

.NET Versions

.NET Framework:

Contains BCL, which is a group of libraries or pre-made classes, so you don't have to build classes from scratch.

Also contains CLR, which is a runtime environment to manage the code.

Its process is to compile C# with the Roslyn compiler to make it an intermediate language, which is platform-independent (can be used on different OSs rather than the one it was compiled in), then this code is compiled by the JIT compiler to the native machine language.

In fact, this still didn't solve the platform-dependent problem which existed before .NET because, at the end of the day, it only works on one OS, which is Windows.

.NET Core:

This version achieved 3 important features, which are:

1- Cross-platform (compile once, use everywhere).

2- Open source, which means many people can see and engage with your work.

3- It was designed as a modular component-based, which leaves you the choice of whether you want to download specific libraries or packages instead of relying on a huge entire framework.

Finally, .NET (only) : they removed the words Core and Framework because they managed to mix both .NET Framework and .NET Core features.

They started this one from .NET 5 version, then kept making a new version every year.

Namespace:

A logical grouping for classes (the compiler sees projects as namespaces).

Solution:

It is the container name in C#.

Connects related projects in the same place.

Reducing Overhead at Runtime (JIT)(just in time):

JIT reduces runtime overhead by compiling frequently used Intermediate Language (IL) code into native machine code and caching it. When the same code is executed again, the compiled native code can be reused instead of being compiled again, reducing the compilation overhead during runtime.